

Omni-Compass: Kubernetes alone, Kubernetes + Omni-Compass, and Omni-Compass direct

Benchmark report, 26 September 2026. Repository: Omni-Compass/The-Omni-Compass-Control-Core-Engine (private), branch claude/kubernetes-clusters-docker-stack-gp26ve. Every number below is produced by code in that repository and can be regenerated; section 21 gives the commands. Each result states whether it was **measured on a live Kubernetes control plane** or **computed in simulation**.

1. Summary

Omni-Compass is a single control engine that senses the whole compute stack and drives its actuators (its "muscles": replica counts, node pools, power caps and others) from one six-state dynamical model, with a safety shield before every action and a kill switch that hands control back. It was compared in three architectures:

- **A. Kubernetes alone.** Kubernetes' own controllers decide: Horizontal Pod Autoscaler (HPA) for replicas, Cluster Autoscaler for nodes; other managers act on their own proposals.
- **B. Kubernetes + Omni-Compass.** Kubernetes' controllers keep running; Omni-Compass governs on top of them (sets the HPA target, gates and sizes the node pool, caps power) as the single authority over their settings.
- **C. Omni-Compass direct.** Omni-Compass is the only decision-maker and actuates the muscles directly; the separate managers no longer decide.

Main result (pre-registered, 1,000 held-out scenarios, simulation). Against Kubernetes alone, B used -28% energy and C -23%; time healthy rose from 79% to 91% (B) and 90% (C); recovery time fell from 58 to 17 and 28 minutes; contradictory commands, pages and human interventions went to zero in both; safety-rule violations fell from 9.5 to 2.7 (B) and 0.0 (C). Of 28 gauges, B is significantly better on 20 and worse on 5; C is better on 16 and worse on 10.

Live Kubernetes result (measured). Two identical Kubernetes clusters (1 control plane + 6 workers) ran the same load at the same time, one without Omni-Compass and one with it. With Omni-Compass: worker nodes in service 6.0 to 3.2; utilisation of the workers in service 0.068 to 0.116. **Energy:** with the parked workers kept on standby, powered and ready (100 W each, the same as idle), energy was 219 vs 218 Wh (-0.46%): parking alone saves essentially nothing; energy per unit of work +8% (not significant; 872 to 944 Wh per core-hour, from minute averages). The -43% first reported for this run holds only if parked workers are powered off. Waiting pods and HPA shortfall were not significantly different. The kill switch restored the original HPA target (50) and all 6 workers. **With every live muscle switched on (section 7.2) the application got slower: p95 response time 486 to 802 ms, energy per unit of work +46% (significant).** The causes were found and fixed across runs 2-4 (section 7.4): response time went from +65% to a tie at p95.

Where Omni-Compass costs something. In the pre-registered study both B and C keep more node-hours powered than Kubernetes alone and start and stop machines more often (more wear), and move the power cap more; C also lets more work wait in the queue and flips scale direction more often. In that study the energy saving comes from power capping and load shaping, not from switching machines off. On the live cluster the saving came from switching machines off. Other limits: the small-cluster release band (section 11); power and heat on the live cluster are modelled, not metered.

Trade-off in one line: B is the strongest all-round result in simulation (energy, health, recovery, queue, coordination and safety all better; wear and node-hours worse); C is the strongest on peak power, heat and safety (zero invariant violations) at the cost of queue length, wear and flip-flops. These are the gauges to tune next.

2. What Omni-Compass is

2.1 The engine

The engine is a six-state ordinary differential equation system, state $x = (E, U, I_U, S, B, B_{\dot{}})$: error E , coherence U , pressure I_U , stress S and a damped bath B . The shipped core (`omnicompass/core.py`) integrates it with fourth-order Runge-Kutta and holds the control input constant across the four stages:

```
(1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,    |u| <= 25
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff = cos(omega_B t / 2) c tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S) = alpha_s S^2/2 + beta_s S^3/4 - delta S
(6) dS/dt = -dPhi/dS
(7) dB/dt = B_dot; dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
(8) R_B[n] = finite-difference audit of (7), never fed back
Controller: u = clip(-f_U(x,t) + 12 (sigma - U), -25, +25)
```

Telemetry (load, queue, power, heat, network, drift, staleness, security) is assimilated into the state each decision; an allocation law turns the state into a demand target ρ^* (the HPA target), a node change, and a power cap. Release of capacity is gated by equation (2): capacity is only released once the control push has converged.

2.2 The nervous system (muscles)

Each muscle has five parts: afferent (pull: sense), the shared engine, efferent (push: act), reflex (the shield checks every push) and kill (hand the muscle back to its own controller). Status in this repository: **wired live on Kubernetes** (sense and push executed through kubectl, each with shield and kill): nodes, HPA target, power cap (in-place CPU limits, enforced by the kernel), security hold, deployment rollouts (pause, resume, undo), batch queue (admit held Jobs); **sensed**: heat (harness heat law on live power, or GPU temperature), network; **hardware connectors** (built and tested with fake hardware, off on CI machines): CPU power states (RAPL read, cpufreq ceiling) and GPU (nvidia-smi power and temperature read, power limit); **open** (registered, no plant yet): memory, storage, cooling, grid, training, inference, agent containment and the rest of the 52-muscle domain map (`docs/DOMAIN_MAP.md`). AI value alignment is explicitly not an Omni-Compass muscle. Code: `omni_controller/controller.py`, `omni_controller/muscles.py`, `omnicompass/nervous.py`.

2.3 The shield and the kill switch

Before any action the shield (`omnicompass/shield.py`) enforces invariants I1 to I5: no expansion during a security block, node count within bounds, step limits, never below the capacity running and pending work needs, and the site power limit. The kill switch (a file or `OMNI_KILL=1`) restores every HPA target Omni-Compass changed from the recorded original, returns the node pool to its native size and drops to observe mode. Every decision and action is written to an append-only audit log.

2.4 Operating modes

Observe (compute and log, write nothing), target (write HPA targets), nodepool (also size a node pool). Laws: `power_protect` (enforce the site power envelope), `throughput` (power envelope not enforced; capacity sized at full power) and `fleet` variants tuned on the 15-second fleet harness.

3. The three architectures, and how each study realises them

Study	A. Kubernetes alone	B. Kubernetes + Omni-Compass	C. Omni-Compass direct	Live or simulated
Pre-registered held-out stack benchmark (2 x 500 scenarios)	k8s_ref_70: documented HPA law (target 0.7, 10% tolerance, 300 s stabilisation) and Cluster Autoscaler (scale-up on backlog, remove after 10 min under 50%), other managers act on their own proposals	omni_k8s_throughput: the same Kubernetes loops, Omni governs on top	omni_direct: Omni senses and actuates the stack directly	simulation
Control-plane replica (24 scenarios)	hpa70_ca: metrics-server, HPA and Cluster Autoscaler replicas at 15 s	omni_target_gate_hpa70_ca: Omni writes the HPA target and gates Cluster Autoscaler scale-down	omni_throughput_full: Omni writes the HPA target, owns node scale-down and power cap; the Cluster Autoscaler may only add nodes	simulation
PlanetLab-shaped demand, fleet plant (8 scenarios)	k8s_hpa70_ca	omni_target: Omni writes the HPA target	omni_fleet: Omni is the node-pool authority, Cluster Autoscaler off	simulation on recorded traces
Live kind cluster, side by side	native: HPA only, 6 workers always on, Omni not running	Omni on top: sets the HPA target and is the sole node-pool authority (cordon, drain, uncordon); Kubernetes' scheduler, kubelet and HPA still execute	not yet built live (section 16)	live

4. Method and why the comparison is fair

- **Frozen before testing.** The allocation law, engine parameters, shield limits, modes, baselines and benchmark code were selected on development seeds (1000, 2000) and frozen, with SHA-256 hashes and program fingerprints in `results/PREREGISTRATION.json`, before any run on the held-out seeds 346410161 and 360555127. `verify.py` re-checks every hash.
- **Same scenarios for every arm.** Each scenario is run under every architecture; comparisons are paired scenario by scenario.
- **Observe-identity check.** Omni-Compass in observe mode must produce a trajectory bit-identical to the arm it observes, or the run is invalid. Held-out: 500 and 500 of 500 identical to the native stack, 500 and 500 of 500 identical to Kubernetes; control-plane replica: 24 of 24; PlanetLab: 8 of 8; live: 0 writes while observing.
- **Statistics.** Differences are paired (Omni minus Kubernetes on the same scenario) with 95% bootstrap confidence intervals. In the held-out study a difference is called better or worse only if the interval excludes zero in the same direction on both independent seeds; 'better in N' counts scenarios. Live results use 2-minute blocks and a bootstrap over blocks.
- **Ablations** show the engine, not an accident of tuning, produces the result (section 5.3).
- **Independent implementations.** A C++ engine, governor, shield and HPA law are checked against the Python ones (section 8).

5. Results: pre-registered held-out benchmark (1,000 scenarios, simulation)

Mean over two held-out seeds x 500 scenarios; verdicts require both seeds to agree.

Gauge	A. Kubernetes alone	B. Kubernetes + Omni-Compass	C. Omni-Compass direct	B vs A	C vs A
ENERGY AND POWER					
Energy per scenario (kWh)	169.1	121.6	129.7	-28% better (better in 991, worse in 9 of 1000)	-23% better (better in 1000, worse in 0 of 1000)

Gauge	A. Kubernet etes alone	B. Kubernetes + Omni-Compass	C. Omni-Com pass direct	B vs A	C vs A
Peak power (kW)	37.6	37.6	28.3	+0.16% not significant (better in 457, worse in 533 of 1000)	-25% better (better in 1000, worse in 0 of 1000)
Node-hours	135.8	140.5	152.4	+3% worse (better in 464, worse in 536 of 1000)	+12% worse (better in 191, worse in 809 of 1000)
Idle node-hours (powered, doing nothing)	46.29	51.73	62.03	+12% worse (better in 446, worse in 554 of 1000)	+34% worse (better in 202, worse in 798 of 1000)
Share of time over the power limit	0.133	0.067	0.018	-50% better (better in 490, worse in 24 of 1000)	-87% better (better in 517, worse in 5 of 1000)
Power-cap travel (how much the cap moved)	0.236	0.887	0.306	+276% worse (better in 37, worse in 963 of 1000)	+30% worse (better in 243, worse in 757 of 1000)
HEAT					
Share of time over the heat limit	0.084	0.026	0.002	-69% better (better in 353, worse in 1 of 1000)	-98% better (better in 354, worse in 0 of 1000)
Thermal travel (how far temperature swung)	0.754	0.632	0.576	-16% better (better in 828, worse in 172 of 1000)	-24% better (better in 993, worse in 7 of 1000)
SPEED AND BACKLOG					
Mean queue (work waiting)	384	217	933	-43% better (better in 218, worse in 515 of 1000)	+143% worse (better in 43, worse in 506 of 1000)
95th-percentile queue (worst moments)	1403	950	2945	-32% better (better in 247, worse in 190 of 1000)	+110% worse (better in 1, worse in 383 of 1000)
Share of time over the backlog limit	0.026	0.015	0.080	-42% better (better in 95, worse in 88 of 1000)	+207% worse (better in 1, worse in 267 of 1000)
RELIABILITY AND RECOVERY					
Availability	0.9990	0.9999	0.9974	+0.09% better (better in 110, worse in 127 of 1000)	-0.16% worse (better in 122, worse in 121 of 1000)
Share of time healthy	0.789	0.913	0.901	+16% better (better in 661, worse in 4 of 1000)	+14% better (better in 659, worse in 4 of 1000)
Share of incidents recovered	0.918	0.996	0.972	+8% better (better in 78, worse in 0 of 1000)	+6% better (better in 54, worse in 0 of 1000)
Recovery time (minutes)	57.8	16.5	27.9	-71% better (better in 602, worse in 0 of 1000)	-52% better (better in 580, worse in 0 of 1000)
Physical SLA breaches (power, heat, backlog)	0.144	0.073	0.089	-49% better (better in 495, worse in 25 of 1000)	-38% better (better in 449, worse in 46 of 1000)
All SLA breaches	0.168	0.073	0.089	-57% better (better in 655, worse in 3 of 1000)	-47% better (better in 611, worse in 37 of 1000)
WEAR AND TEAR					
Machines started	4.8	6.6	9.4	+37%	+95%
Machines stopped	1.2	3.3	2.7	+185%	+130%
Node start/stop events	6.0	9.9	12.1	+66% worse (better in 56, worse in 907 of 1000)	+102% worse (better in 199, worse in 782 of 1000)
Machine round trips (stopped then restarted)	1.07	1.73	2.70	+61% worse (better in 119, worse in 464 of 1000)	+152% worse (better in 26, worse in 866 of 1000)
Scale reversals (flip-flops)	2.20	1.93	5.45	-13% better (better in 474, worse in 199 of 1000)	+147% worse (better in 86, worse in 803 of 1000)
CONTROL QUALITY AND SAFETY					
Contradictory commands between managers	9.46	0.00	0.00	-100% better (better in 666, worse in 0 of 1000)	-100% better (better in 666, worse in 0 of 1000)
Safety-rule (invariant) violations	9.50	2.71	0.00	-72% better (better in 664, worse in 1 of 1000)	-100% better (better in 669, worse in 0 of 1000)
Invariant violations excluding power	6.86	0.00	0.00	-100% better (better in 666, worse in 0 of 1000)	-100% better (better in 666, worse in 0 of 1000)
Security violations	0.889	0.000	0.000	-100% better (better in 118, worse in 0 of 1000)	-100% better (better in 118, worse in 0 of 1000)
Pages to on-call	0.584	0.000	0.000	-100% better (better in 92, worse in 0 of 1000)	-100% better (better in 92, worse in 0 of 1000)
Human interventions required	0.284	0.000	0.000	-100% better (better in 92, worse in 0 of 1000)	-100% better (better in 92, worse in 0 of 1000)

5.1 What the numbers say

- **Energy:** B -28%, C -23% versus Kubernetes alone, although both keep slightly more node-hours powered. The saving comes from power capping and load shaping: time over the power limit halves (B) or nearly vanishes (C), and C cuts peak power by a quarter. Releasing idle machines, which the live cluster showed, is not what drives this study; idle node-hours are a gauge to tune.
- **Reliability:** time healthy and recovery improve sharply in both B and C; the share of incidents recovered rises.
- **Coordination:** separate managers issue contradictory commands (A); a single authority issues none (B, C). Pages and human interventions go to zero because Omni-Compass acts on the conditions that would have paged someone.
- **Heat:** C cuts time over the heat limit the most, because it governs power caps and load together.
- **Costs:** node start/stop events rise (B +66%, C +102%), machine round trips rise, the power cap moves more, and C's queue and scale reversals are higher than Kubernetes alone. The 24-scenario control-plane study (section 6) shows the opposite for wear (fewer machine stops), so wear depends on the plant and law and is a primary tuning target. These are reported, not tuned away.

5.2 Architecture A has a hidden cost: the fragmented stack without Kubernetes

For reference, the stack with each manager acting alone and no Kubernetes loops (native) used 131.9 kWh, was healthy 39% of the time, took 127 minutes to recover and produced 56.7 contradictory commands and 28.1 invariant violations per scenario. Kubernetes already improves on that; Omni-Compass improves on Kubernetes.

5.3 Ablations: is it the engine?

Comparison	Metric	Mean difference	95% CI (seed 1)	95% CI (seed 2)
Omni direct vs omni_direct_no_dynamics	energy_kwh	-1.954	[-2.061, -1.775]	[-2.158, -1.834]
Omni direct vs omni_direct_no_dynamics	invariant_violations	+0.000	[+0.000, +0.000]	[+0.000, +0.000]
Omni direct vs omni_direct_no_dynamics	mean_queue	+39.200	[-4.742, +71.297]	[+7.307, +76.997]
Omni direct vs omni_direct_no_dynamics	recovery_minutes	+0.950	[-0.090, +2.650]	[-0.410, +2.260]
Omni direct vs omni_direct_no_dynamics	time_healthy	-0.000	[-0.004, +0.002]	[-0.002, +0.003]
Omni direct vs omni_direct_no_dynamics	violation_heat	-0.002	[-0.004, -0.000]	[-0.005, -0.000]
Omni direct vs omni_direct_no_shield	energy_kwh	-0.759	[-0.864, -0.642]	[-0.875, -0.660]
Omni direct vs omni_direct_no_shield	invariant_violations	-0.674	[-0.750, -0.592]	[-0.760, -0.596]
Omni direct vs omni_direct_no_shield	mean_queue	+121.945	[+90.814, +149.893]	[+100.054, +152.127]
Omni direct vs omni_direct_no_shield	recovery_minutes	+1.535	[+0.560, +3.210]	[+0.480, +2.600]
Omni direct vs omni_direct_no_shield	time_healthy	+0.001	[-0.001, +0.003]	[-0.000, +0.003]
Omni direct vs omni_direct_no_shield	violation_heat	-0.000	[-0.000, +0.000]	[-0.001, +0.000]

'no_dynamics' runs the same allocation law with equations (1) to (7) frozen; 'no_shield' removes the shield. The difference is full engine minus ablation: negative energy means the evolving dynamics save energy the frozen engine does not; the shield ablation shows what the shield prevents.

6. Results: Kubernetes control-plane replica (24 scenarios, simulation)

Replicas of the documented metrics-server, HPA (15 s) and Cluster Autoscaler (10 s) loops, with Karpenter-lite as a second Kubernetes reference; Omni-Compass decides every 300 s.

Gauge	A. Kubernetes alone (HPA + Cluster Autoscaler)	Reference: HPA + Karpenter-lite	B. Kubernetes + Omni-Compass (HPA target + CA gate)	C. Omni-Compass authority (HPA target, nodes, power cap)	C vs A (95% CI, wins of 24)
Energy (kWh)	168.8	161.7	155.6	122.3	-28% better [-49.724, -43.292], 24 of 24
Peak power (kW)	38.7	39.5	40.8	32.8	-15% better [-8.962, -2.584], 17 of 24
Time over power limit	0.050	0.050	0.048	0.015	-70% better [-0.051, -0.019], 14 of 24
Time over heat limit	0.045	0.047	0.045	0.012	-73% better [-0.048, -0.018], 13 of 24
Time healthy	0.930	0.937	0.931	0.983	+6% better [+0.033, +0.073], 18 of 24
Recovery time (min)	1.857	1.982	3.430	0.430	-77% better [-1.948, -0.906], 15 of 24
Physical SLA breaches	0.031	0.032	0.031	0.009	-71% better [-0.032, -0.013], 14 of 24
Mean queue	0.204	0.204	1.011	0.683	+235% worse [+0.168, +0.836], 0 of 24
Machines started	21.8	24.9	25.5	20.0	-8% better [-3.542, -0.250], 4 of 24
Machines stopped	13.2	17.0	24.1	3.833	-71% better [-11.917, -7.083], 24 of 24
Scale reversals	1.333	1.917	2.000	1.000	-25% better [-0.667, -0.083], 4 of 24
Invariant violations	0.000	0.000	0.000	0.000	0% not significant [+0.000, +0.000], 0 of 24
Pages to on-call	0.000	0.000	1.792	0.458	new not significant [+0.000, +1.375], 0 of 24

7. Results: live Kubernetes (measured)

7.1 Side by side, two identical clusters at the same time

kind clusters (real Kubernetes API server, scheduler, kubelet, HPA and metrics-server), 1 control plane + 6 workers each; the official php-apache HPA workload (target 50); the same stepped load (1, 2, 3, 1, 2, 1 load generators over 20 minutes after a 2-minute warm-up); capture every 15 s. Native: Omni-Compass not started. Omni: nodepool mode, HPA target, node pool, power sensing. Run 36209933218, 2026-09-26.

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
Duration (min)	20.10	20.13	
Worker nodes in service, mean	6.00	3.20	-46.7%
Worker nodes in service, min	6.00	3.00	-50.0%
CPU used (cores), mean	0.819	0.745	-9.1%
CPU allocatable (cores), mean	12.00	6.40	-46.7%
Utilisation	0.068	0.116	+70.5%
Workers powered (in service + standby)	6.00	6.00	0
Node-hours in service	2.01	1.07	-46.6%
Power (W), mean, parked workers on standby at 100 W	657	654	-0.5%

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
Power (W), peak, parked workers on standby at 100 W	691	687	-0.6%
Energy (Wh), parked workers on standby at 100 W	219	218	-0.5%
Energy (Wh), parked workers in low-power standby at 50 W	219	171	-22%
Energy (Wh), parked workers in deep sleep at 10 W	219	133	-39%
Energy (Wh), parked workers powered off (as first reported)	220	125	-43%
Pending pods, pod-minutes	0.567	0.267	-52.9%
Pending pods, peak	1	1	0
HPA replicas, mean	8.71	6.20	-28.8%
HPA replicas, peak	10	8	-20.0%
HPA shortfall, minutes	0.833	1.07	+28.0%

Metric (per unit of work, 2-minute blocks)	Kubernetes alone	Kubernetes + Omni-Compass	Change	95% CI	Verdict
Wh per core-hour, standby at 100 W (2-minute blocks from minute averages)	872	944	+8.2%	[-150, +292]	not significant
Node-hours in service per core-hour	7.8535	4.6976	-40.2%	[-5.0982, -1.1072]	fewer (not energy while parked nodes stay powered)
Utilisation	0.0694	0.1224	+76.3%	[+0.0217, +0.0830]	better
Pending-pod minutes per hour	1.7253	0.8333	-51.7%	[-3.4961, +1.6667]	not significant
HPA shortfall minutes per hour	2.5458	3.3399	+31.2%	[-3.3277, +5.0066]	not significant

Omni-Compass decisions: nodes per minute 5 4 3 then 3 for the remaining 17 minutes; HPA target 78-79% (Kubernetes alone: 50%). Node-pool resizes: 3 (three workers cordoned and drained, their pods rescheduled by Kubernetes). Kill switch: HPA target restored to 50, 6 of 6 workers back in service.

7.2 Live, side by side, every live muscle (run 36213152881 (2026-09-26 02:54-03:19 UTC))

Same two-cluster setup; the Omni arm drives HPA target, node pool, power cap (in-place pod CPU limits), heat (harness law on live power), security hold, rollout guard. Real response times: HTTP requests to the app timed every 5 s on both clusters. **This run made the application slower.**

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
Worker nodes in service, mean	6.00	3.20	-46.7%
Node-hours	2.01	1.07	-46.8%
Power (W), mean, standby counted	664	643	-3.2%
Power (W), peak	720	664	-7.7%
Energy (Wh), standby counted	222	215	-3.4%
CPU used (cores), mean	0.908	0.593	-34.6%
Utilisation	0.076	0.093	+22.6%
Energy per core-hour (Wh)	731	1,083	+48.1% worse
Pending pods, pod-minutes	1.70	0	-100%
HPA replicas, mean	9.09	5.00	-45.0%

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
HPA replicas, peak	10	5	-50.0%
Requests timed	955	850	-11.0%
Response time (ms), mean	260	415	+59.7% worse
Response time (ms), median	230	320	+39.2% worse
Response time (ms), 95th percentile	486	802	+64.8% worse
Response time (ms), 99th percentile	675	1,113	+64.7% worse
Failed requests (%)	0	0	0

Metric (2-minute blocks)	Kubernetes alone	+ Omni-Compass	Change	95% CI	Verdict
kWh per core-hour	0.7596	1.1107	+46.2%	[+0.1901, +0.5099]	worse
Node-hours per core-hour	6.8941	5.6440	-18.1%	[-2.7717, +0.4577]	not significant
Utilisation	0.0766	0.0964	+25.8%	[-0.0016, +0.0394]	not significant

Actions: hpa target writes 1, node pool resizes 3, power cap pod resizes 7, rollout actions 0, power cap per minute 0.695 every minute (the law floor), hpa target 77%, nodes 5 4 3 then 3. Kill switch: restored target 50; workers 6 of 6; pod CPU limits restored.

Diagnosis: The engine saw the cluster as idle (node utilisation about 7%) and had no response-time signal, so it capped the app pods at the law floor and packed replicas to a 77% target (5 instead of 10). The app pods were busy, so responses slowed. Fix wired: latency afferent (p95 over a declared 500 ms SLO enters as queue pressure) and a power-cap reflex (never below pod usage x 1.3); re-run pending.

7.3 Other live runs

Run	Setup	Result
live-kind, run 36205408388	1 node; baseline 10 min (Omni observing) then Omni target mode 10 min	0 writes while observing; kill switch restored 50; pending-pod minutes -48.6% (significant, but the baseline phase included warm-up); energy per core-hour no significant difference (one node cannot be parked)
live-kind-full, run 36205869009	1 control plane + 3 workers	node pool never resized: 3 of 3 every minute at about 8% utilisation, because the fleet law needs more than 3 nodes of slack; kill switch restored target and workers
live-kind-full, run 36207925928	1 control plane + 6 workers; sequential baseline then full engine	nodes in service 6 to 5 to 4 to 3; node-hours in service per core-hour -38.2%, utilisation +63.7% (significant); the reported kWh per core-hour -35.8% counted parked workers as powered off; with parked workers on standby the saving largely disappears; kill switch restored target 50 and 6 of 6 workers

7.4 Live runs after the first all-muscle run

Run	Change tested	Validity	Result
36214629046 (2026-09-26 03:23:03:48 UTC), commit c182a63	latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap usage reflex	valid	p95 481 to 600 (+24.6% worse); p99 596 to 777 (+30.3% worse); energy 222 to 219 (-1.5%)
36216647786 (2026-09-26 04:02:04:28 UTC), commit 052b705	SLO reflex; eviction receipt fixed	INVALID as a test of the engine	p95 479 to 320 (-33.2%); p99 583 to 420 (-28.0%); energy 224 to 220 (-1.7%)
36218637030 (2026-09-26 04:42:05:08 UTC), commit 47d0353	controller fail-safe; benchmark prints controller log and rejects early stops	PARTIAL	p95 491 to 492 (+0.3% (tie)); p99 645 to 683 (+5.8% worse); energy 223 to 221 (-1.1%)

Run 3 is recorded as invalid: a refused Kubernetes call stopped the controller after 3 of 20 decisions and the cluster held Omni's last settings with no engine running. The fix is a fail-safe: a failed decision is skipped; three in a row restore native settings and stop the controller. In run 4 the fail-safe did exactly that at minute 16, and the audit showed why: the least-privilege role allowed writing a pod's CPU limit but not reading it first, which kubect! does. In runs 3 and 4 no power cap was ever applied. Over run 4's 16 governed minutes node-hours per unit of work fell 24% and utilisation rose 36% (both significant); median response time -10%, p95 a tie, p99 +6% (not significant). The permission is fixed (with a receipt) and a run in which the fail-safe fires is now rejected; the re-run is in progress.

8. Engineering verification

`python verify.py` runs 89 checks (0 failures in the recorded log `results/VERIFY_LOG.txt`), including:

- reference engine SHA-256
- core parity vs reference engine and 500 fixtures
- C++ core vs 500 fixtures (max abs 3.55e-15)
- C++ governor vs Python governor (throughput)
- C++ shield vs Python shield (enforce and violations)
- negative control: parity test detects a C++ shield that no longer blocks rollouts during a security block
- independent C++ HPA replica law vs fleet harness HPA
- soak test (throughput), 100,000,000 decisions, no failures, memory flat after warm-up (2899 ns per decision, RSS checkpoints [3840, 3840, 3840, 3840] kB)
- held-out seed 346410161: observe mode identical to native
- live controller against a fake cluster: observe writes nothing, target bounded, kill restores, node pool bounded and dry-run safe

Also tested: the live controller against a fake cluster (observe writes nothing; targets bounded; kill restores from the HPA annotation after a restart; node pool bounded and dry-run safe), the full-engine options (parked and control-plane nodes excluded; the kill switch restores the node pool exactly once) and pilot scoring (detects a real gain, reports no gain on identical clusters, detects a service regression).

9. Physics and models

- **Stack plant power:** each node draws idle 0.38 kW plus 1.12 kW x utilisation; a power cap throttles delivered capacity.
- **Heat:** thermal state follows a first-order lag toward $0.34 + 0.62 \times \text{power stress}$ (time constant about 7 steps); heat above 0.82 throttles capacity; 'over the heat limit' means thermal above 1.03.
- **Live kind cluster:** kind nodes have no power meter, so power is a declared model: 100 W idle + 150 W x CPU utilisation per worker in service, plus a standby power for each parked (cordoned and drained) worker. Standby defaults to the idle power (the worker stays powered and ready); lower values apply only to a declared sleep state, zero only to machines really powered off. The first live reports counted parked workers as zero; section 7.1 gives both. The same constants drive the governor's power sense and the energy score, so they cannot disagree. On real hardware this is replaced by metered power (RAPL, PDU or BMC).
- **Savings model** (results/SAVINGS.csv): a 1,000-node web cluster at 0.4 kW per node, PUE 1.4, \$0.12/kWh and 0.4 kg CO2/kWh; reduction versus HPA 0.7 + Karpenter-lite of 14% to 20% (fleet plant) gives roughly 710 to 960 MWh, \$85,000 to \$115,000 and 280 to 380 t CO2 per year.
- **Engine overhead:** about 2.9 microseconds per decision in C++, memory flat over 100 million decisions.

10. What is proven, what is simulated, what is not claimed

Claim	Status	Evidence
Engine equations are finite and deterministic; C++ equals Python	Proven	verify.py, 500 fixtures, max error 3.6e-15
Observe mode changes nothing	Proven (simulation and live)	bit-identical trajectories; 0 writes live
Kill switch restores native control	Proven (simulation and live)	HPA target 50 and all workers restored live
Omni-Compass acts on a real Kubernetes control plane (HPA target, node pool)	Proven live	section 7
Fewer nodes in service than Kubernetes with a fixed node pool, same load served	Measured live	section 7.1
Lower energy on the live cluster	Not shown while parked nodes stay on standby; -22% to -43% only if parked nodes sleep or power off	section 7.1
Better energy, health, recovery, coordination than Kubernetes (HPA + CA)	Pre-registered simulation	section 5
Better than Karpenter-lite on energy	Simulation	sections 6, and PlanetLab fleet plant
Better than upstream Karpenter or Cluster Autoscaler binaries, live	Not yet tested	section 12
Metered energy savings on physical servers	Not yet tested	power is modelled
GPU power-limit muscle saves 15-19% energy with <1% slower responses	Simulation calibrated to metered H100 data (MLPerf)	section 12
CPU frequency muscle	Simulation (uncalibrated): about -3.5% energy, -67% heat	section 12
Tuned laws remove wear and node-hour negatives (C-throughput)	Pre-registered amendment, new held-out data	section 13
Cooling, grid, memory, storage and the other open muscles	Not claimed	no plant or connector yet
Makes AI models aligned or trustworthy	Not claimed	value alignment is outside Omni-Compass

11. Limits and threats to validity

- Simulated studies use documented-behaviour replicas of Kubernetes controllers, not the upstream binaries; the Kubernetes reference omits Karpenter consolidation, VPA, scheduling constraints and disruption budgets.
- The live cluster is kind: nodes are containers on one CI machine; the two live arms ran on two machines at the same time, so machine-to-machine variation is part of the noise; each live arm is 20 minutes, one repetition.
- Live power and heat are modelled; parked kind workers are drained containers. If parked machines must stay on standby, parking reduces nodes in service but not energy; live energy savings then have to come from power caps, CPU power states and heat control, which are not yet wired live.
- The live native arm had no node autoscaler, so its node pool was always full; the fair live opponent is Karpenter or Cluster Autoscaler (section 16).
- The live significance for energy per core-hour with standby power is computed from minute averages (10 two-minute blocks), not from the 15-second capture.
- The fleet law releases a node only when the pool has more than three nodes of slack; a three-worker pool cannot scale down (observed live, reproduced offline). Small clusters need a pool-size-aware release band.
- Architecture C was measured in simulation only; the live C (Kubernetes' controllers parked, Omni-Compass as the only brain) is not built yet.
- Service quality differences in the live runs (pending pods, HPA shortfall) are not statistically significant at this run length.

12. GPU and CPU muscles: device plant calibrated to metered hardware (simulation)

The GPU power-limit and CPU frequency muscles cannot be actuated on CI machines (no GPU; the hypervisor hides RAPL and cpufreq). Their connectors are built (section 2.2) and this plant shows what they do. **GPU calibration:** the performance-versus-power-limit exponent is fitted to MLPerf Inference v4.0 results for an NVIDIA DGX-H100 (8 x H100-SXM, 700 W TDP), MaxQ (power-limited with `nvidia-smi -pl`, the same command the Omni-Compass GPU connector sends) versus MaxP, system power metered by a Yokogawa WT333E; Apache 2.0. Fitted exponent 0.353 to 0.490 (median 0.449); all three are run. **CPU:** a standard first-order model (dynamic power ~ frequency cubed) against a schedutil-style governor; not calibrated to metered data yet. **S** is a fixed manual 70% cap, what an operator could do by hand. 24 scenarios, 8 load families; * = paired 95% interval excludes zero.

Vessel	Gauge	A. Native	S. Fixed 70% cap	B. + Omni-Compass	C. Omni-Compass direct
cpu_web	Energy (kWh)	115.169	103.342 (-10%*)	110.899 (-4%*)	111.238 (-3%*)
cpu_web	Energy per unit of work	0.672	0.614 (-9%*)	0.650 (-3%*)	0.652 (-3%*)
cpu_web	95th-pct response time (x baseline)	1.000	6.577 (+558%*)	1.040 (+4%*)	1.040 (+4%*)
cpu_web	Minutes over service target	0.000	70.458 (new*)	0.167 (new)	0.167 (new)
cpu_web	Minutes over heat limit	26.708	0.000 (-100%*)	8.875 (-67%*)	8.958 (-66%*)
cpu_web	Peak power (kW)	33.954	20.475 (-40%*)	34.074 (+0.35%)	34.074 (+0.35%)
cpu_web	Minutes near full power	8.000	0.000 (-100%*)	10.167 (+27%*)	10.167 (+27%*)
gpu_mlperf_median	Energy (kWh)	142.255	117.038 (-18%*)	119.254 (-16%*)	119.254 (-16%*)
gpu_mlperf_median	Energy per unit of work	0.824	0.679 (-18%*)	0.691 (-16%*)	0.691 (-16%*)
gpu_mlperf_median	95th-pct response time (x baseline)	1.000	1.521 (+52%*)	1.007 (+0.70%*)	1.007 (+0.70%*)
gpu_mlperf_median	Minutes over service target	0.000	7.125 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_median	Minutes over heat limit	22.458	0.000 (-100%*)	6.125 (-73%*)	6.125 (-73%*)
gpu_mlperf_median	Peak power (kW)	38.403	28.288 (-26%*)	37.089 (-3%*)	37.089 (-3%*)
gpu_mlperf_median	Minutes near full power	6.500	0.000 (-100%*)	6.208 (-4%)	6.208 (-4%)

Vessel	Gauge	A. Native	S. Fixed 70% cap	B. + Omni-Compass	C. Omni-Compass direct
gpu_mlperf_least_favourable	Energy (kWh)	142.255	118.430 (-17%*)	120.770 (-15%*)	120.770 (-15%*)
gpu_mlperf_least_favourable	Energy per unit of work	0.824	0.688 (-17%*)	0.699 (-15%*)	0.699 (-15%*)
gpu_mlperf_least_favourable	95th-pct response time (x baseline)	1.000	1.628 (+63%*)	1.008 (+0.80%*)	1.008 (+0.80%*)
gpu_mlperf_least_favourable	Minutes over service target	0.000	8.333 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_least_favourable	Minutes over heat limit	22.458	0.000 (-100%*)	6.708 (-70%*)	6.708 (-70%*)
gpu_mlperf_least_favourable	Peak power (kW)	38.403	28.345 (-26%*)	37.532 (-2%*)	37.532 (-2%*)
gpu_mlperf_least_favourable	Minutes near full power	6.500	0.000 (-100%*)	6.417 (-1%)	6.417 (-1%)
gpu_mlperf_most_favourable	Energy (kWh)	142.255	113.878 (-20%*)	115.645 (-19%*)	115.645 (-19%*)
gpu_mlperf_most_favourable	Energy per unit of work	0.824	0.661 (-20%*)	0.670 (-19%*)	0.670 (-19%*)
gpu_mlperf_most_favourable	95th-pct response time (x baseline)	1.000	1.300 (+30%*)	1.006 (+0.55%*)	1.006 (+0.55%*)
gpu_mlperf_most_favourable	Minutes over service target	0.000	5.375 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_most_favourable	Minutes over heat limit	22.458	0.000 (-100%*)	5.250 (-77%*)	5.250 (-77%*)
gpu_mlperf_most_favourable	Peak power (kW)	38.403	28.133 (-27%*)	36.174 (-6%*)	36.174 (-6%*)
gpu_mlperf_most_favourable	Minutes near full power	6.500	0.000 (-100%*)	5.833 (-10%*)	5.833 (-10%*)

Reading: on GPUs Omni-Compass saves 15-19% energy across the measured range with response time 0.6-0.8% slower; a fixed cap saves about the same energy but slows responses 30-63% and misses the service target. On CPUs the native governor already tracks demand, so frequency control alone saves about 3.5%; its main effect is heat (-67%). CPU frequency control is below the 10% bar and is a physical ceiling of that muscle, not a tuning gap.

13. Amendment: tuned laws, frozen, then tested on new held-out data (simulation)

Two law variants were tuned on the development seeds only (1000, 2000; tuning/SEARCH* . json), frozen with SHA-256 hashes in tuning/PREREGISTRATION_AMENDMENT_2026-09-26 . json and pushed (commit b79d1dd, 03:17 UTC) before a single run on new held-out seeds 731001 and 731002 (2 x 500 scenarios). The frozen engine files are unchanged; the original pre-registered result (section 5) stays the primary result. B-wear: node release held longer. C-throughput: Omni-Compass direct under the throughput law with a wider release band and engine-gated cap. + better, ! worse (both seeds agree), blank not significant.

Gauge	A. Kubernetes alone	B_frozen	B_wear	C_frozen	C_throughput
energy_kwh	170.001	122.670 (-28%) +	123.552 (-27%) +	130.042 (-24%) +	123.353 (-27%) +
node_start_stop	6.119	10.070 (+65%) !	8.210 (+34%) !	11.985 (+96%) !	3.065 (-50%) +
machine_round_trips	1.031	1.724 (+67%) !	0.831 (-19%) +	2.711 (+163%) !	0.863 (-16%) +
scale_reversals	2.176	1.931 (-11%) +	1.805 (-17%) +	5.394 (+148%) !	1.838 (-16%) +
node_hours	136.812	141.488 (+3%) !	144.688 (+6%) !	152.143 (+11%) !	122.531 (-10%) +
idle_node_hours	46.662	52.152 (+12%) !	55.353 (+19%) !	61.218 (+31%) !	35.788 (-23%) +
mean_queue	438.529	246.346 (-44%) +	246.324 (-44%) +	1103.704 (+152%) !	1029.828 (+135%) !
p95_queue	1462.260	1005.114 (-31%) +	1005.044 (-31%) +	3185.168 (+118%) !	2684.025 (+84%) !
violation_backlog	0.030	0.016 (-45%) +	0.016 (-45%) +	0.085 (+182%) !	0.064 (+115%) !
recovery_minutes	61.470	16.805 (-73%) +	16.805 (-73%) +	28.925 (-53%) +	37.700 (-39%) +

Gauge	A. Kubernetes alone	B_frozen	B_wear	C_frozen	C_throughput
power_cap_travel	0.249	0.894 (+259%) !	0.893 (+259%) !	0.307 (+23%) !	0.697 (+180%) !
invariant_violations	9.625	2.748 (-71%) +	2.747 (-71%) +	0.000 (-100%) +	0.000 (-100%) +
sla_violation_total	0.171	0.076 (-55%) +	0.076 (-55%) +	0.094 (-45%) +	0.145 (-15%) +
pages	0.671	0.000 (-100%) +	0.000 (-100%) +	0.000 (-100%) +	0.000 (-100%) +
time_healthy	0.785	0.909 (+16%) +	0.909 (+16%) +	0.896 (+14%) +	0.837 (+7%) +
availability	0.999	1.000 (+0.09%) +	1.000 (+0.09%) +	0.997 (-0.23%) !	0.996 (-0.32%) !
recovered	0.907	0.997 (+10%) +	0.997 (+10%) +	0.971 (+7%) +	0.963 (+6%) +

Worse gauges: B_frozen 5, B_wear 4, C_frozen 10, C_throughput 5.

14. Response time: can one mode beat Kubernetes on every gauge? (simulation, development seeds)

The fleet-mode constants that drove the live runs were selected (before this work) by a rule that scored energy, finished work and backlog, but never waiting time. A response-time gauge was added to the fleet plant (fleet/sim_slo.py: M/M/c queueing delay per workload plus backlog drain; the frozen trace is reproduced exactly). Measured with it, fleet mode cut energy 31% on web services while p95 response time went from 132 ms to about 20 s. That is where the live slowdown came from: its HPA target floor (rho_min = 76.9%) packs pods too hot for latency-sensitive services.

A speed-first law (omnicompass/speed.py; the engine equations unchanged) separates the two jobs the frozen law gave one number: pods run at a latency-safe target while machines are sized to what the pods request. 400 configurations were searched against HPA 70% + Cluster Autoscaler with the rule that no gauge may be worse in any development scenario. None passed, for three measured reasons: (1) several gauges are already at their physical floor (100% work done, zero violations, batch p95 = pure service time), so a tie is the best any controller can do; (2) on the GPU vessel demand exceeds the hardware, so every arm is saturated; (3) energy, response time and machine churn trade two-of-three, because the autoscaler's idle slack is both where the energy is and what absorbs the 90-second boot delay. Examples, mean change vs Kubernetes: fleet mode energy -31% with far worse response time; speed #176 energy -1%, p99 -41%, churn 3.6x; speed #159 p99 -48%, churn -27%, energy +16%. Source: tuning/SPEED_FINDINGS_2026-09-26.md.

15. Named products: OpenShift, Google GKE, Azure AKS, IBM Turbonomic (simulation, development seeds)

Each product is emulated from its documented behaviour on the same fleet plant (not the vendors' binaries): OpenShift's documented ClusterAutoscaler example (threshold 0.4, unneeded 5 min, delay after add 10 min); GKE optimize-utilization (MostAllocated packing, more aggressive scale-down; declared as threshold 0.65, 2 min, because Google publishes no numbers); AKS node auto-provisioning (Karpenter, WhenEmptyOrUnderutilized, consolidateAfter 0 s); Turbonomic (container requests resized every 10 min to p99 per-pod usage, its default aggressiveness; nodes suspended toward 0.7 packing). Means over web services, 4 development seeds:

Gauge (web)	Kubernetes (GKE balanced)	OpenShift	GKE optimize	AKS NAP	Turbonomic	Omni fleet mode	Omni speed #159
energy (kWh)	12.19	12.53	10.4	9.779	10.08	8.392	15.9
p95 (ms)	131.6	131.6	132.2	132.7	125.1	2.034e+04	112.3
p99 (ms)	3,095	2,815	3,947	4,295	4.94e+04	7.634e+04	129.8

Gauge (web)	Kubernetes (GKE balanced)	OpenShift	GKE optimize	AKS NAP	Turbonomic	Omni fleet mode	Omni speed #159
machine starts+stops	10.5	8.5	14.75	19	11.5	13.25	6.75
scale reversals	1.5	0.75	3.5	6	1.5	4.25	1

Every product sits on the same energy / response-time / churn triangle; none wins all three. Karpenter-style consolidation (AKS NAP) saves about 20% energy with a worse p99 tail and about twice the machine churn; Turbonomic's p99 resizing interacts with the HPA and inflates the tail; OpenShift's documented settings sit close to upstream. Omni's speed mode leads on response time and churn at an energy cost; its fleet mode leads on energy at a large response-time cost.

16. The problem-map muscles: all nine built and tested on held-out data (simulation)

Each open row of the industry problem map (section 18) is now a muscle in omnilab/, with the strongest native tool as opponent, Omni-Compass, and Omni-Compass with its engine equations not evolved (to show what the equations themselves add). Constants were chosen on development seeds 1-8, every file was frozen by SHA-256 (results/muscles/PREREGISTRATION.json), then 30 held-out seeds were run once. Verdicts from a paired bootstrap 95% interval.

Muscle (map row)	Strongest native opponent	Omni better	Omni worse	Tie / not significant
Right-sizing, HPA+VPA conflict (1, 2)	hpa_vpa	cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57%	p95 ms +15%; oom kills +531%	work done
Cold start (5)	keda	p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8%	cold starts +736%	-
GPU packing (6)	binpack	energy kwh -2%; idle gpu hours powered -20%	wait mean min +16%; migrations 0 to 6.07	wait p95 min, frag blocked min, jobs done
Training power swings (7)	floor_safe	energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6%	max ramp mw s +127%	ramp violation s
GPU failures and stragglers (8)	detect	goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99%	-	checkpoint overhead pct
Cooling (10)	reset	pue -2%; cooling mwh -13%; inlet violation min -73%	-	max inlet c
LLM inference, KV cache (12)	keda	ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56%	-	ttft p50 s, tpot p95 ms
Runaway AI agents (13)	static	rogue overspend usd -96%; time to contain min -99%; peak subagents -59%	false stops +357%; honest work pct -7%	forbidden executed
Energy attribution in VMs (11)	ratio	attr error pct -37%; worst vm error pct -34%	-	total error pct

Each entry is the plain change of Omni-Compass against the opponent (for example p99 -99% means the slow tail is 99% shorter; goodput +10% means 10% more useful training time). Worse entries are real costs, not rounding: right-sizing trades a slightly slower typical response (p95 +15%, both far under the 500 ms target) and a few more memory kills for a 99% shorter tail; cold start keeps fewer idle instances than KEDA's 5-minute cooldown, so more requests meet a cold start, but users wait far less because KEDA polls every 30 s; GPU packing powers idle GPUs off sooner and jobs wait a few seconds longer; containment throttles honest agents in their legitimate bursts (about 7% of their work) and stops about two honest agents a day, against a 96% cut in runaway spend and containment in minutes instead of hours. Against the other native arms

(the default tools most teams run) the wins are larger; every comparison is in results/muscles/*_HELDOUT.json.

What the engine itself contributes. Across the nine muscles the full engine and the engine-not-evolved arm differ by 1.0 percentage points on average per gauge. The mapping from engine state to action carries most of each result; the evolved dynamics mainly smooth. In power smoothing an arm where the bath equation (7) alone sets the site's draw, with no ramp rule, cut the steepest ramp 63% but did not hold the grid's limit: the human sets the boundary, the engine operates inside it.

17. Every negative, its cause and its status

Negative	Where	Status	Cause	What would fix it
Node start/stop cycles (wear)	B, pre-registered	reduced (+65% to +34%), not removed	node release thresholds are fixed numbers	hybrid: Kubernetes serves the queue, Omni releases nodes with the C-throughput law (removed wear there: -50%)
Node start/stop, round trips, reversals	C, pre-registered	fixed in C-throughput (new held-out)	power-protect law released nodes too eagerly	adopted in C-throughput
Node-hours and idle node-hours	B and C, pre-registered	fixed in C-throughput (-10%, -23%); not in B	power capping trades lower watts for more servers on	hybrid as above; node-aware cap
Queue / backlog / availability -0.3%	C (both variants)	UNRESOLVED	not the replica law and not the sizing constants (both tested); likely the delayed observation or direct-mode proposals	trace one scenario step by step; candidate: feed the queue into the engine without the extra delay
Power-cap movement	B and C	inherent	moving the cap is how capping saves energy; freezing it removed most of the saving (tested: -28% to -2%/-7%)	none needed: electronic setting, no physical wear; reported
CPU frequency saving 3.5%	device plant	physical ceiling	the native Linux governor already follows demand	value is in heat (-67%) and in combining with power caps
GPU node on/off saving 3%	fleet plant	superseded	training nodes cannot be switched off	the GPU power-limit muscle (15-19%)
Live response time +65% (p95) and energy per unit of work +46% with every muscle	live kind, run 36213152881	reduced to p95 +25% (run 2), tie in run 4	no response-time afferent; cap held at the law floor; HPA target floor 76.9%	latency afferent, cap reflex, SLO reflex; speed-first law (section 14)
Fleet mode p95 20 s vs 132 ms on web services	fleet plant with response-time gauge	cause found; speed-first law built	fleet constants were selected without a response-time gauge; HPA target floor 76.9%	choose the mode per service: speed-first where latency matters
No mode better than Kubernetes on every gauge in every scenario	fleet plant, 400 configurations	not achievable as posed	gauges at physical floors; hardware-bound GPU vessel; energy / response time / churn trade two-of-three	anticipation (pre-adding machines before the daily rise) is the one untested mechanism that could break the trade
Controller stopped after one refused call; power cap never applied under least privilege	live runs 3 and 4	fixed	one exception ended the loop; the role lacked get on pods/resize	fail-safe restore after 3 failures; permission and receipt added; runs with a fail-safe rejected
Right-sizing p95 +15%, more memory kills than VPA	omnilab rightsize, held-out	open, small	Omni sizes CPU closer to demand; VPA's 8-hour p90 memory keeps more slack	larger memory margin (costs memory-hours)
More cold starts than KEDA	omnilab coldstart, held-out	trade-off	shorter keep-alive than KEDA's 5-minute cooldown	longer keep-alive where cold starts matter more than instance-hours
GPU jobs wait +16% (seconds) vs Volcano binpack	omnilab gpupack, held-out	trade-off	idle GPUs powered off sooner	longer power-off delay
Honest agents throttled (-7% work) and ~2 false stops a day vs static caps	omnilab containment, held-out	trade-off	throttling on the engine's integrated need catches legitimate bursts	per-agent declared burst budgets; human approval before stop

Negative	Where	Status	Cause	What would fix it
Engine dynamics add little beyond the mapping	all nine muscles	reported	the mapping from state to action carries the effect	wire the engine's control effort (equation 2) directly as the actuator command and test it
Live energy with parked servers on standby ~0%	live kind	open	parked servers still draw standby power	live power cap and CPU/GPU muscles; sleep states where hardware allows
Small clusters (<= 3 workers) never release	live kind	open	fleet law release band of 3 nodes	pool-size-aware release band (law change)
Pages in the 24-scenario replica	control-plane replica	open	longer queue triggers the page rule	same as queue

18. The industry problem map: what Omni-Compass is aimed at

One engine; the vessel (the plant it sits on) is the only thing that changes. Industry figures are approximate, from the public sources named, and are context, not results of this report. Status: **live** = measured on a real Kubernetes control plane; **sim** = demonstrated in this repository's simulations; **open** = mapped, connector not built.

Problem	Scale in the industry (approximate, source)	Best software today	Omni-Compass vessel and muscles	Gauge that shows it	Status
Data-centre electricity growth	about 415 TWh in 2024, about 1.5% of world electricity, projected near 945 TWh by 2030 (IEA, Energy and AI, 2025)	Karpenter, Cluster Autoscaler, CAST AI, Spot Ocean; Kepler for metering	compute vessel: nodes, HPA, power cap	energy, node-hours, idle node-hours	sim; live only where parked nodes can sleep or power off
Idle and over-provisioned capacity	Kubernetes clusters commonly run near 10-15% average CPU utilisation (CAST AI and Datadog industry reports); roughly a quarter to a third of cloud spend reported as waste (Flexera State of the Cloud)	VPA, Goldilocks, StormForge, Kubecost/OpenCost	compute vessel: nodes, HPA; requests and memory (omnilab/rightsize.py)	utilisation, node-hours per core-hour, core- and GiB-hours	live + sim
Controllers fighting each other	documented conflicts, e.g. HPA and VPA on the same CPU metric (Kubernetes documentation advises against it)	none: each tool decides alone	single authority over replicas and requests	contradictory commands, scale reversals, OOM kills	sim, held-out (section 16)
Outages and slow recovery	most significant outages cost over \$100,000 (Uptime Institute annual outage analysis)	Argo Rollouts, Flagger, SRE runbooks, AIOps (Dynatrace, Datadog)	compute vessel + deployments (partial)	time healthy, recovery time, SLA breaches	sim
On-call load and alert fatigue	widely reported burnout in SRE surveys	PagerDuty, alert tuning	all muscles: act before the page	pages, human interventions	sim
Heat and cooling limits	cooling is a large share of facility energy; average PUE about 1.5 (Uptime Institute survey)	DCIM (Schneider EcoStruxure), DeepMind cooling AI (reported about 40% less cooling energy)	heat (sensed), cooling plant (omnilab/cooling.py)	PUE, cooling energy, inlet violations	sim, held-out (section 16)
Site power and grid-connection limits	multi-year waits for new grid connections are widely reported	Meta Dynamo power capping, Intel RAPL	power cap (wired), batteries and demand response (open)	peak power, time over power limit	sim
GPU energy and power limits	GPU fleets widely reported well below full utilisation; H100 TDP 700 W	NVIDIA DCGM and MIG, Run:ai, Kueue; manual MaxQ power limits	GPU power-limit muscle (hardware connector)	energy per unit of work, response time, heat	sim, calibrated to MLPerf metered H100 data: -15% to -19% energy, +0.6-0.8% response time

Problem	Scale in the industry (approximate, source)	Best software today	Omni-Compass vessel and muscles	Gauge that shows it	Status
Batch deadlines and fair sharing		Kueue, Volcano, Slurm	batch muscle (live: admit held jobs); GPU packing (omnilab/gpupack.py)	queue wait, fragmentation	live wired + sim, held-out
Hardware wear	power cycling and churn shorten component life	none as a governed objective	nodes: start/stop cycles and reversals	machines started and stopped, round trips	mixed: better in the 24-scenario study, worse in the held-out study; tuning target
Carbon reporting and reduction	regulatory disclosure is expanding	Google carbon-aware computing, Kepler	carbon-aware placement (open)	kWh and CO2 per unit of work	sim (modelled)
Runaway AI agents and spend	~\$10,000 overnight examples (Dark Reading)	per-tool quotas and permissions	agent containment muscle (omnilab/containment.py)	rogue spend, time to contain, false stops	sim, held-out (section 16)

19. What comes next

- Live architecture C: park HPA, VPA, Cluster Autoscaler and Karpenter; Omni-Compass sets replicas, resources, placement, priorities and quotas directly; Kubernetes keeps execution and reflexes (restarts, rescheduling); the kill switch wakes the parked controllers.
- Live opponent at full strength: Karpenter (kwok provider) and Cluster Autoscaler in architecture A.
- 24 live scenarios (traffic, failures, power and heat limits, batch and AI, growth, mixed) with repetitions.
- More muscles two-way: CPU power states, memory, batch queues, network, security, then GPU and cooling on hardware.
- Metered power on physical machines.

20. Questions and answers

Does Omni-Compass replace Kubernetes? No. Kubernetes keeps running containers, placing pods, restarting failures and networking. Omni-Compass replaces the separate decision loops (how many replicas, how many nodes, what power) with one authority. In architecture C Kubernetes becomes one muscle.

What happens if Omni-Compass crashes or is switched off? The kill switch restores every setting it changed and returns control to Kubernetes' own controllers; this was exercised live and in simulation. A crashed controller writes nothing further.

Can it make things worse? Every action passes the shield first; it never goes below the capacity running and pending work needs, and never changes more than the step limit. In the held-out benchmark it had fewer safety violations than Kubernetes. Its real costs are listed in sections 1 and 5.1.

How fast does it decide, and what does it cost to run? One decision per 60 s on live Kubernetes (300 s in the replica), with a 15 s fast path that adds nodes for pending pods. The engine takes about 2.9 microseconds per decision.

Why does it save energy? In the pre-registered stack study mainly by power capping and load shaping (fewer minutes over the power limit, lower peak), while keeping slightly more machines on. On the live cluster it took machines out of service (6 to 3 workers) and packed replicas more densely; that saves energy only if the parked

machines sleep or power off. With parked machines on standby the live saving was about zero, so live savings must come from power caps and CPU power states.

Does it slow applications down? It can, in the energy-first fleet mode: its HPA target floor packs pods too hot for latency-sensitive services (section 14). Live, the first all-muscle run was slower (p95 +65%); after the fixes run 4 was a tie at p95 and 10% faster at the median. For latency-sensitive services the speed-first mode is the right setting.

Can Omni-Compass control AI agents? It controls what an agent can touch, spend and do, and how fast; not what the model thinks. In the containment muscle (section 16) it cut runaway spend 96% and contained runaways in minutes instead of hours, with a least-privilege identity that blocks forbidden actions outright; the cost is some throttling of honest agents' bursts.

Does the engine hold everything in its basin by itself? Not against an outside limit it is not told. The bath equation alone smoothed training power ramps 63% but did not keep them under the grid's limit; with the human-set limit as the boundary it held it with zero violations. The human sets the boundaries; the engine operates inside them.

Is this tuned to the test? Parameters were selected on development seeds and frozen with hashes before the held-out seeds were run; verify.py fails if any frozen file changes.

How many scenarios and how certain? 1,000 pre-registered held-out scenarios, 24 control-plane scenarios, PlanetLab traces and live runs; 95% bootstrap intervals, two independent seeds must agree.

What is measured versus modelled? Live: node counts, replicas, pods, CPU, the controller's actions and the kill switch. Modelled: power and heat everywhere, and everything in the simulated studies.

Is the mathematics sound? The core is a closed six-state system integrated with RK4; the C++ and Python implementations agree to 3.6e-15 on 500 reference trajectories; 100 million decisions ran without a non-finite value.

Who owns it and how can it be used? The Omni-Compass LLC. Free for evaluation, research and non-commercial use; commercial use requires a paid licence; protected by copyright and by patents and patent applications (see LICENSE and NOTICE).

What is not claimed? Superiority over upstream Karpenter or Cluster Autoscaler live, metered savings on physical hardware, GPU or facility control on real hardware, the vendor products' own binaries (they are emulated from documentation), and anything about AI value alignment.

How do I check it myself? Run the commands in section 21; the live runs are GitHub Actions workflows in the repository.

21. Reproduce

```
pip install -r requirements.txt
python verify.py                                # all checks, hashes, parity, soak
python benchmarks/stack_benchmark.py ...       # held-out stack benchmark (see HARNESS.md)
python -m k8s_controlplane.benchmark --scenarios 24 --seed 424242
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 --out /tmp/pl
GitHub Actions: benchmark (live side by side), live-kind-full, live-kind
python tools/full_report.py ... && python pilot/bench_pdf.py docs/BENCHMARK_REPORT.md docs/BENCHMARK_REPORT.pdf
```

22. Glossary

- **HPA**: Horizontal Pod Autoscaler: Kubernetes controller that sets replica counts from CPU utilisation versus a target.
- **Cluster Autoscaler, Karpenter**: Kubernetes add-ons that add and remove nodes.
- **Node, worker**: a machine (here a container in kind) that runs pods.
- **Cordon, drain**: mark a node unschedulable, then move its pods elsewhere.
- **kind**: Kubernetes in Docker: a real Kubernetes control plane whose nodes are containers.
- **Observe mode**: Omni-Compass computes and logs but writes nothing.
- **Invariant**: a safety rule the shield enforces before any action.
- **Paired bootstrap CI**: resampling the per-scenario differences to get a 95% interval for the mean difference.
- **Pre-registration**: freezing code and parameters, with hashes, before running the test data.